RoadGuard — AI Pothole Detection Project Report

Project type: Computer-vision prototype for road-safety inspection
Implementation: Python, JupyterLab, OpenCV, Ultralytics YOLO, Linux
Target environment: Linux and LinuxONE-ready workflow
Project name: RoadGuard

Abstract

RoadGuard is an AI-assisted pothole-detection project that helps road-inspection teams screen road images for visible potholes. The project combines a reproducible Linux/Jupyter computer-vision pipeline with a browser demonstration interface. The notebook downloads and prepares public road-damage data, converts annotations to a YOLO-compatible format, fine-tunes a one-class object detector, evaluates the model, and runs image, video, or webcam inference. The web demo communicates the same workflow visually through a real pothole image, an automatic detection-loop video, an interactive scan animation, and a review-oriented output card.

RoadGuard is designed as an inspection-support tool. It does not claim to measure pothole depth, determine structural risk, make a repair decision, or replace a qualified human inspector.

1. Problem statement

Potholes create safety and maintenance problems for drivers, pedestrians, public-works departments, and transport agencies. They may contribute to vehicle damage, loss of control, traffic disruption, and delayed maintenance. Traditional inspection often depends on manual surveys or public reports. These approaches can be slow, inconsistent, and difficult to scale across a large road network.

The central problem addressed by RoadGuard is:

> How can a road authority screen large volumes of road imagery for visible potholes and create consistent visual evidence for human review?

The project focuses on the first stage of the workflow: identifying likely visible pothole regions in road imagery. It does not attempt to automate the complete engineering or maintenance lifecycle.

2. Objectives

The project objectives are to:

1. Prepare a reproducible pothole dataset for Linux and JupyterLab.
2. Convert Pascal VOC annotations into YOLO object-detection labels.
3. Train a one-class pothole detector using transfer learning.
4. Evaluate the detector with standard object-detection metrics.
5. Run inference on images, video frames, and optional webcam input.
6. Produce annotated evidence that can be reviewed by an inspector.
7. Communicate the workflow through a polished, accessible browser prototype.
8. Document a responsible path toward batch processing on LinuxONE.

3. Proposed solution

RoadGuard uses a pretrained YOLO object detector and fine-tunes it for one target class: `pothole`. The end-to-end flow is:

1. Download the official RDD2022 India road-damage archive.
2. Extract Pascal VOC XML annotations and source images.
3. Retain the RDD2022 `D40` class as the pothole target.
4. Convert each selected bounding box to normalized YOLO format.
5. Create a dataset YAML file and train a pretrained YOLO model.
6. Validate the model on the prepared validation split.
7. Run prediction on new images, a folder, a video, or a webcam.
8. Save annotated frames and metrics for human review.

The browser demo represents this flow without pretending that the visual animation is a live benchmark result. It shows the input road frame, scanning state, highlighted region, review label, and human-verification limitation.

4. Dataset and preparation

The default data source is the RDD2022 road-damage dataset, using the India country archive. The included `scripts/prepare_rdd2022.py` script downloads the archive, extracts it under

`data/raw/`, reads Pascal VOC XML annotations, keeps the pothole class `D40`, and writes a one-class YOLO dataset under `data/processed/rdd2022_pothole/`.

The repository does not include downloaded datasets. This keeps the GitHub repository small and respects the need to review dataset redistribution terms. The downloader and attribution links are included so that another user can reproduce the preparation step in their own Linux environment.

Pothole-600 is documented as an optional additional source for later experiments. Any additional dataset must be checked for licensing, annotation format, class definitions, and geographic bias before it is merged into training.

5. Model and implementation

The notebook uses Python and the Ultralytics YOLO Python interface. A pretrained checkpoint is loaded and fine-tuned on the pothole dataset. The important experiment settings are visible in the notebook, including:

- image size;
- number of epochs;
- batch size;
- device selection;
- confidence threshold; and
- output paths.

The device selection prefers CUDA when a compatible GPU is available and otherwise uses CPU. The workflow is therefore suitable for a Linux Jupyter environment, although training time depends on the available processor, memory, dataset size, and image resolution.

OpenCV is used for video-oriented workflows. The notebook includes examples for a local video source and an optional webcam source. The output is saved as annotated images or video frames rather than treated as a final maintenance decision.

6. Browser demonstration

The `web_demo/` folder contains a self-contained static Node.js site. It has no database and no required application API. The main interface contains:

- a RoadGuard product header and problem statement;
- a real pothole image as the input frame;

- a muted, autoplaying, looping video showing the scan line and green pothole marking;
- a **Detect pothole** button with animated progress;
- a visible bounding box and the exact label **“Pothole detected — review required”**;
- plain-English explanation of the output;
- benefits for faster screening, prioritization, consistent evidence, and maintenance planning; and
- a human-verification disclaimer.

The loop video is a product-demonstration asset. It is intentionally separate from the notebook’s measured inference results. This distinction prevents the UI prototype from being mistaken for a reported accuracy score.

7. Evaluation plan

The notebook runs validation and exposes metrics including precision, recall, mAP50, and mAP50-95. These metrics must be recorded only after the notebook has completed successfully. The repository does not invent benchmark values because the final result depends on the chosen compute environment, training duration, dataset version, split, and model checkpoint.

For a meaningful operational evaluation, the following should be measured on a held-out and geographically representative test set:

- precision and recall for visible potholes;
- mAP50 and mAP50-95;
- false positives per frame and per kilometer of road imagery;
- missed detections under rain, night, glare, blur, and occlusion;
- processing speed on the intended LinuxONE or edge environment; and
- time from image capture to a maintenance-ready review record.

A threshold should be selected based on the cost of missed potholes versus the cost of additional human reviews. A high-confidence prediction should still remain a review signal rather than an autonomous repair authorization.

8. LinuxONE relevance

RoadGuard is a Linux-native workload that can be packaged in a virtual environment or container and operated on LinuxONE. A production-oriented design could ingest frames from

survey vehicles or smartphones, run inference in batches, and write results to an auditable object store, CSV, GIS layer, or maintenance-management system.

LinuxONE is relevant to the project because it can provide a centrally managed enterprise Linux environment for repeatable dependencies, access control, monitoring, scaling, and high availability. Multiple teams or cities could use the same service while keeping the model version, data lineage, and review records auditable.

The current prototype does not claim that it uses a proprietary IBM accelerator. It demonstrates a Linux-ready AI workload and a practical integration path for LinuxONE.

9. Benefits and stakeholders

The primary stakeholders are public-works departments, municipal road authorities, transport agencies, road-maintenance contractors, and inspection teams. Drivers and pedestrians benefit indirectly when dangerous road defects are identified and prioritized earlier.

Potential benefits include:

- faster screening of image collections;
- more consistent visual evidence for work-order discussions;
- better prioritization of field inspections;
- support for survey-vehicle and smartphone imagery;
- a repeatable notebook workflow for retraining with local data; and
- a clear human-in-the-loop operating model.

These are intended workflow benefits, not guaranteed outcome measurements. They should be validated with a pilot using real local inspection procedures.

10. Limitations and responsible use

The detector is limited to visible patterns in images. It does not estimate physical depth, structural damage, drainage condition, repair cost, ownership, legal responsibility, or the urgency of a road closure. Performance may decrease with night scenes, rain, glare, motion blur, unusual camera angles, partial obstruction, or road surfaces that differ from the training distribution.

Potential risks include false positives that consume inspection time, missed potholes that create false reassurance, geographic bias, and overreliance on confidence scores. Mitigations include local data collection, geographically held-out testing, threshold calibration, periodic monitoring,

clear uncertainty communication, audit logs, and mandatory human verification before maintenance action.

11. Reproducibility instructions

From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python scripts/prepare_rdd2022.py --root data
jupyter lab
```

Open `notebooks/AI_Potholes_Detection.ipynb` and run the cells in sequence. The web demo can be started separately:

```
cd web_demo
npm start
```

The repository intentionally ignores datasets, generated runs, Python virtual environments, model weights, and exported ONNX files. These artifacts should be stored separately when their licenses and storage policy allow.

12. Conclusion

RoadGuard presents a practical and responsible approach to AI-assisted pothole screening. The Jupyter pipeline makes dataset preparation, model training, evaluation, and inference inspectable. The browser demo makes the intended user experience easy to understand: road image in, candidate pothole region out, human review next.

The next technical step is not simply to train a larger model. It is to collect representative local imagery, measure performance on a held-out geographic split, calibrate thresholds with inspection teams, and integrate verified detections into a secure maintenance workflow.

References

1. RDD2022 project repository: https://github.com/sekilab/RoadDamageDetector
2. RDD2022 Figshare record: https://figshare.com/articles/dataset/RDD2022_-_The_multi-national_Road_Damage_Dataset_released_through_CRDDC_2022/21431547
3. Pothole-600 project page: https://sites.google.com/view/pothole-600/dataset
4. Ultralytics Python usage: https://docs.ultralytics.com/usage/python/
5. Ultralytics repository: https://github.com/ultralytics/ultralytics
6. OpenCV: https://opencv.org/
7. Jupyter: https://jupyter.org/